Mobile Demo Script Explanation -- UPOS (Reversing, Challenge 06)

Olawoyin Joseph Pamilerin, ISU520196

In one sentence: The app's 'randomness' is a fixed-seed LFSR -- completely predictable once reverse-engineered, which is what makes offline flag recovery possible.

Background -- what the terms below mean

What does "Reversing" mean, as a category? A **Reversing** challenge gives you someone else's already-built app (an APK) and asks you to figure out, by reading its code, some secret value it's hiding -- a PIN, a password, a flag string.

What is an APK, and what does "decompile" mean? An APK is the installable file for an Android app. It ships as compiled code that isn't directly readable, so we **decompile** it -- turn it back into readable, Java-like source code -- using a free tool called **jadx** (opened as **jadx-gui**).

Why is there no exploit app to build here? Reversing challenges usually aren't solved by writing an Android app at all -- they're solved by reading the decompiled logic and then writing a small script (usually Python) that re-implements or defeats that same logic on a regular computer, offline.

How the APK was obtained

The target APK was **downloaded directly from the challenge's own page** on challs.reyammer.io, before doing anything else. A copy of it is kept at:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Reversing/06 UPOS\Original Target APK.apk
```

How the solution was built

No Android app was built for this one at all. It was solved by **decompiling the target with jadx** and then writing a small **Python script** (or, in a couple of cases, working the logic out by hand) that re-implements or defeats the exact same check on a regular computer, offline. The recovered value was then typed directly into the challenge's own flag box.

The vulnerability / technique, in plain terms

com.mobisec.upos.FC.checkFlag() validates the flag with SHA256(flag) == a hardcoded hex hash. The real character-recovery mechanism is a Streamer class: a textbook Linear Feedback Shift Register (LFSR) with a hardcoded seed ("01101000010", tap 8) -- there is no real randomness. For each of 30 positions, the LFSR selects a (row, col) cell in a 256x256 matrix (assets/lotto.dat) whose value is always a perfect square; the integer square root, split into two bytes, is a Caesar-shifted (+7 mod 26) version of the two real flag characters. Undoing the shift recovers them. The one real trap: 12 'decoy' Streamer.step() calls are scattered through checkFlag() before the main 30-iteration loop, silently advancing the LFSR state -- miss those and every recovered character comes out as garbage. A further 5 step() calls later in the method are provably dead code (the method always throws before reaching them), and several substring checks ("this_is_", "really_", anti-Frida /proc scanning) are genuine but unenforced decoys. Ported the LFSR, matrix lookup and inverse-Caesar logic to Python; verified the LFSR port was byte-for-byte correct by compiling and running Streamer.java's own

embedded main() self-test and diffing the output.

Files to open

1. Open the target APK in jadx-gui:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Reversing/06 UPOS\Original Target APK.apk
```

2. Streamer.java (the LFSR):

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Reversing/06 UPOS\Decompiled Sources (jadx)\com\mobisec\
```

In jadx-gui's left tree: Source code -> com -> mobisec -> upos -> Streamer.java

3. upos_solver.py (the recovery script):

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Reversing/06 UPOS\Solver Script\upos_solver.py
```

What to say -- explanation of the exploit, step by step

1. I decompiled the APK with jadx and read FC.checkFlag(), finding it validates the flag with SHA256(flag) against a hardcoded hash.

2. I found the real character-recovery mechanism: a Streamer class implementing a fixed-seed Linear Feedback Shift Register (LFSR) with no real randomness.

3. I traced how the LFSR selects cells in a 256x256 perfect-square lookup table (assets/lotto.dat), and how each cell's square root, split into two bytes, is a Caesar-shifted version of two real flag characters.

4. I ported this logic to Python, verifying my LFSR port was byte-for-byte correct by compiling and running the app's own Streamer.java self-test.

5. My first attempt produced garbage until I found 12 "decoy" step() calls hidden earlier in checkFlag() that silently advance the LFSR state; adding those fixed the recovery.

6. I computed the SHA-256 of my recovered flag, confirmed it matched the hardcoded hash exactly, then submitted it to the live flag box, where it was accepted.

If an emulator isn't available

If asked how you got/confirmed the flag: "I decompiled the target with jadx, recovered the secret value with a script (no exploit app was built or uploaded for this one), and typed the recovered flag directly into the challenge site's own flag box." There is no APK submission or platform analysis log for this challenge -- don't claim one.

Result

The flag this challenge produced:

```
MOBISEC{Isnt_this_a_truly_evil_undebuggable_piece_of_sh^W_software??}
```

Source: Recovered by pure static analysis, verified by exact SHA-256 match against the hash hardcoded in the decompiled app, then submitted to the live CTFd flag box and accepted

The real, verbatim output this run produced:

```
$ python upos_solver.py
Recovered core (61 chars): 'Isnt_this_a_truly_evil_undebuggable_piece_of_sh^W_software??'
```

Full flag: MOBISEC{Isnt_this_a_truly_evil_undebuggable_piece_of_sh^W_software??}

Flag length: 69 (expect 69)

SHA256(flag): 4193d9b72a5c4805e9a5cc739f8a8fc23b2890e13b83bb887d96f86c30654a12

expected: 4193d9b72a5c4805e9a5cc739f8a8fc23b2890e13b83bb887d96f86c30654a12

MATCH: True